



Electrical and Computer Engineering Department

Communication Systems ENEE3309

DPCM Predictor Assignment

Student 1 Name:	Belal Hammad	Student 1 ID:	1240859
Student 2 Name:	Zaid Sabha	Student 2 ID:	1241068
Student 3 Name:	Wadee Fatafta	Student 3 ID:	1250758

Section: 2

Instructor: Dr. Qadri Mayala

Date: 13/9/2026

Part A - First order Prediction

$$x[n] = \{10, 12, 15, 17, 18, 20, 21, 23\}$$

$$\hat{x}[n] = x[n-1]$$

A1. $\hat{x}[n] = \{-, 10, 12, 15, 17, 18, 20, 21\}$

* since $\hat{x}[n] = x[n-1]$ requires a previous sample, the predictor is undefined at $n=0$ (there's no $x=-1$). therefore, $\hat{x}[0]$ and $e[0]$ will be left blank

A2. $e[n] = \{-, 2, 3, 2, 1, 2, 1, 2\}$

$$\boxed{\text{rang} = \text{max} - \text{min}}$$

A3. $\text{Range}_{x[n]} = 23 - 10 = \underline{13}$ From 10 to 23

$\text{Range}_{e[n]} = 3 - 1 = \underline{2}$ From 1 to 3

A4. To quantize a signal properly, the number of levels used must be enough to cover its full dynamic range. Since $e[n]$ has a small range (2) compared to $x[n]$ (range 13), it needs far fewer quantization levels to achieve the same step size, and therefore fewer bits per sample.

Part B - Effect of the Predictor Coefficient

$$\hat{x}[n] = a x[n-1]$$

$$x[n] = \{10, 12, 15, 17, 18, 20, 21, 23\}$$

For $a = 0$

starting
at $n=1$

$$\hat{x}[n] = \{0, 0, 0, 0, 0, 0, 0\}$$

$$e[n] = \{12, 15, 17, 18, 20, 21, 23\}$$

$$e^2[n] = \{144, 225, 289, 324, 400, 441, 529\}$$

$$\sum e^2 = 2352$$

$$MSE = \frac{1}{N-1} \sum e^2$$

$$= \frac{2352}{7} = \boxed{336}$$

For $a = 0.5$

$$\hat{x}[n] = \{5, 6, 7.5, 8.5, 9, 10, 10.5\}$$

$$e[n] = \{7, 9, 9.5, 9.5, 11, 11, 12.5\}$$

$$e^2[n] = \{49, 81, 90.25, 90.25, 121, 121, 156.25\}$$

$$\sum e^2 = 708.75$$

$$MSE = \frac{708.75}{7} = \boxed{101.25}$$

For $\alpha = 1$

$$\hat{X}[n] = \{10, 12, 15, 17, 18, 20, 21\}$$

$$e[n] = \{2, 3, 2, 1, 2, 1, 2\}$$

$$e^2[n] = \{4, 9, 4, 1, 4, 1, 4\}$$

$$\sum e^2[n] = 27$$

$$MSE = \frac{27}{7} = \boxed{3.86}$$

For $\alpha = 1.2$

$$\hat{X}[n] = \{12, 14.4, 18, 20.4, 21.6, 24, 25.2\}$$

$$e[n] = \{0, 0.6, -1, -2.4, -1.6, -3, -2.2\}$$

$$e^2[n] = \{0, 0.36, 1, 5.76, 2.56, 9, 4.84\}$$

$$\sum e^2 = 23.52$$

$$MSE = \frac{23.52}{7} = \boxed{3.36}$$

4. the smallest MSE is when $a = 1.2$
 $MSE = \underline{3.36}$

5. When $a = 0$, there is no prediction at all
 $e[n] = x[n]$

Prediction error carries exactly same information and dynamic range as the original signal, so we are quantizing $x[n]$ directly with no compression.

6. When the coefficient is excessively large, the predictor exceeds the true value of the signal, so the error no longer stays small and around zero. It grows in magnitude and can reach large negative values. This means the quantizer ends up handling wider dynamic range than the original signal itself, which is the opposite of what DPCM is meant to achieve.

7. No, prediction only reduces error's amplitude when a suitable coefficient is used. e.g:

at $a = 0 \rightarrow e[n] = x[n]$ zero reduction in amp.

at $a = 0.5 \rightarrow$ range of $e[n]$ (7 to 12.5), larger than $a = 1$ error range

~~So a reduction~~

So a good reduction in amp. is only achieved by

finding the optimum coefficient.

Part C - Finding the Optimum Predictor

$$a_{\text{opt}} = \frac{\sum x[n]x[n-1]}{\sum x^2[n-1]}$$

To find optimum a we need to calculate minimum

$$\text{MSE} = \frac{\sum e^2}{N-1}$$

since $(N-1)$ is const. $\rightarrow$ calculate min. $\sum e^2$

$$F(a) = \sum e^2$$

$$= \sum (x[n] - \hat{x}[n])^2$$

$$= \sum (x[n] - ax[n-1])^2$$

$$= \sum (x^2[n] - 2ax[n]x[n-1] + a^2x^2[n-1])$$

to find min. val. we diff. with respect to a and set the result to zero:

$$0 = \sum (0 - 2x[n]x[n-1] + 2ax^2[n-1])$$

$$0 = -\sum x[n]x[n-1] + \sum ax^2[n-1]$$

$$a_{\text{opt}} = \frac{\sum x[n]x[n-1]}{\sum x^2[n-1]}$$

a -opt for sequence

$$X_{[n]} = \{10, 12, 15, 17, 18, 20, 21, 23\}$$

$$X_{[n-1]} = \{-, 10, 12, 15, 17, 18, 20, 21\}$$

$$a\text{-opt} = \frac{\sum x_{[n]} x_{[n-1]}}{\sum x_{[n-1]}^2} \quad n=1 \rightarrow N-1$$

$$= \frac{12(10) + 15(12) + \dots}{(10)^2 + (12)^2 + \dots}$$

$$a\text{-opt} = \frac{2124}{1923} = 1.105$$

$$\Rightarrow \hat{X}_{[n]} = \{11.05, 13.26, 16.575, 18.785, 19.89, 22.1, 23.205, 25.415\}$$

$$e_{[n]} = \{0.95, 1.74, 0.425, -0.785, 0.11, -1.1, -0.205\}$$

$$e_{[n]}^2 = \{0.902, 3.027, 0.181, 0.616, 0.012, 1.21, 0.042\}$$

$$\sum e^2 = 5.99$$

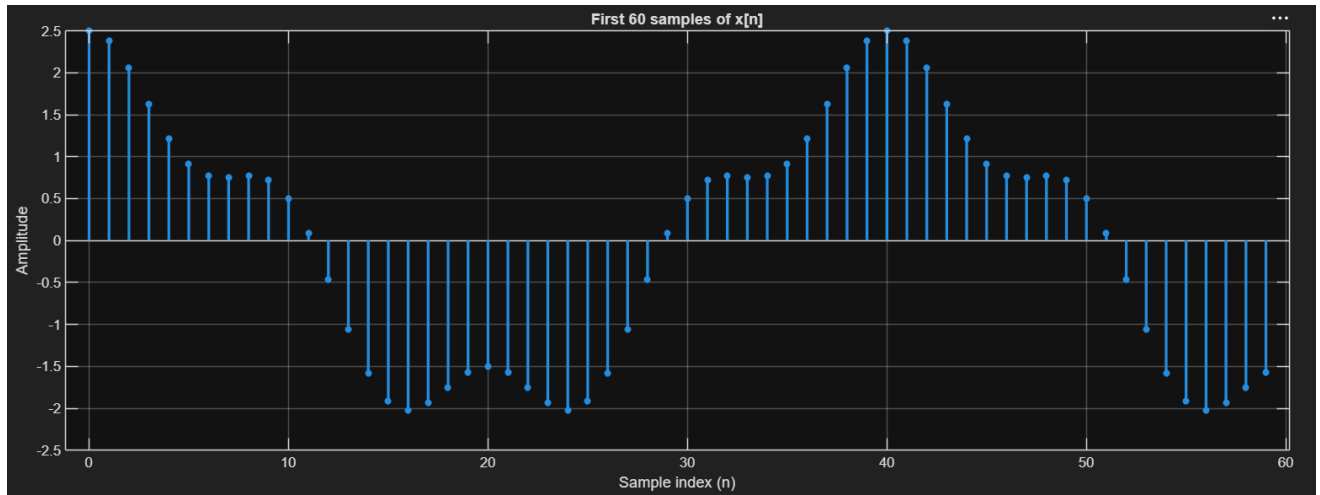
$$MSE = \frac{5.99}{7} = 0.856$$

a	0	0.5	1	1.2	1.105 (a -opt)
MSE	336.	101.25	3.857	3.36	0.856

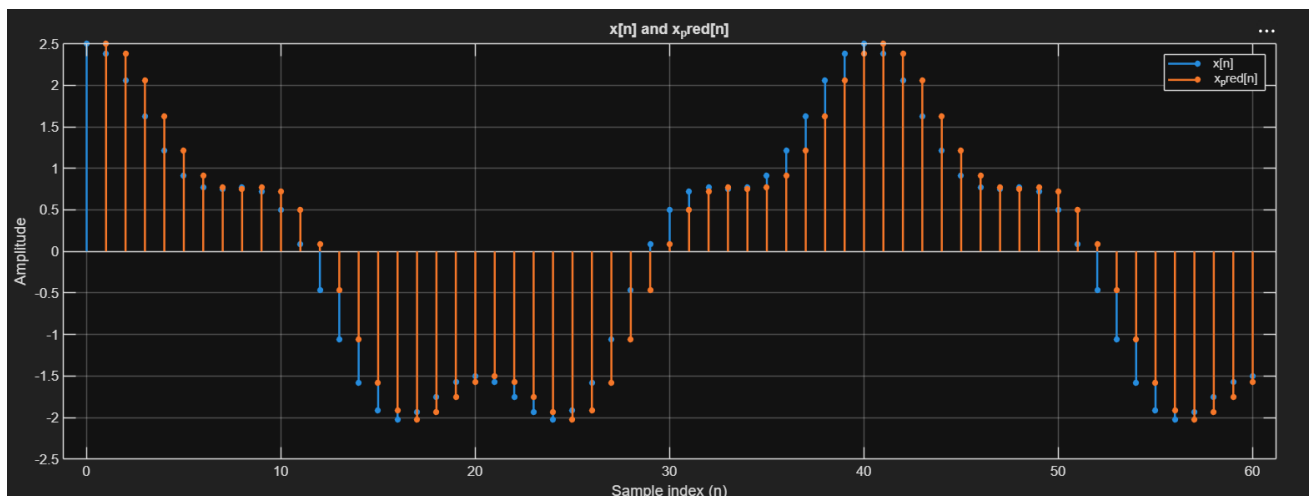
As we can see a -opt beats every value tested in Part B.

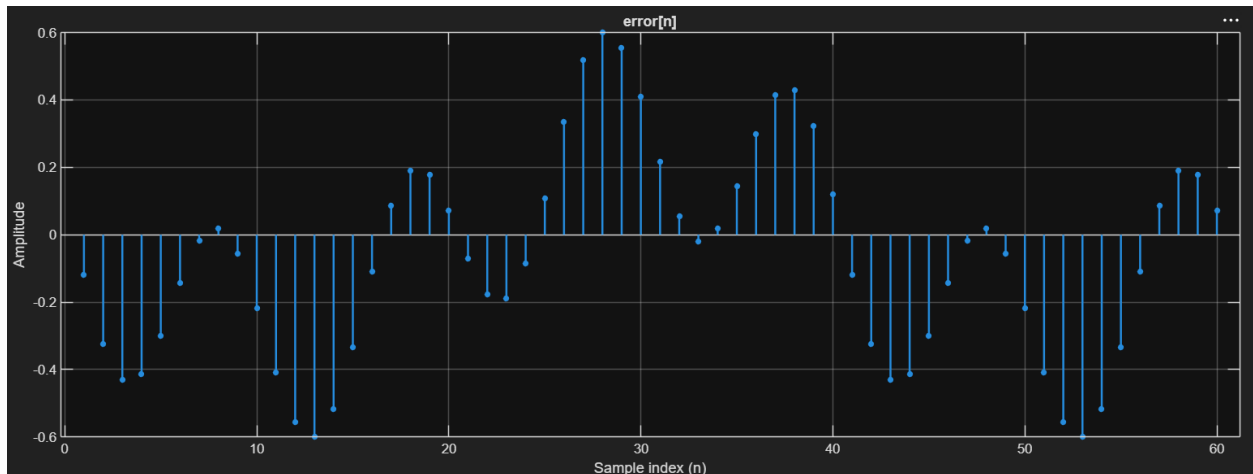
Part D – MATLAB Investigation

D1 – Generate and plot signals



D2 – Simple Predictor





Question: From the figures, explain why $e[n]$ is generally smaller than $x[n]$.

Answer: because the signal has significant correlation between adjacent samples, x and x_{pred} are close to each other, which reduces the difference between them, thus reducing the value of $e[n]$.

D3 – Prediction Gain

After calculating P_x , P_e , G_p , $G_p(\text{db})$ by MATLAB code we got these values:

```

46
47 %===== D3 =====
48
49 N = length(x);
50
51 Px = (1/N) * sum(abs(x).^2);
52 Pe = (1/(N - 1)) * sum(abs(e(2:end)).^2);
53
54 Gp = Px/Pe;
55
56 Gp_db = 10 * log10(Gp);
57
Command Window
Px = 2.125000
Pe = 0.097407
Gp = 21.815616
Gp(db) = 13.387675
>>

```

Question: Calculate the prediction gain. What does a large prediction gain mean?

Answer:

Prediction Gain turned out to be $21.81 = (13.39 \text{ db})$.

Prediction Gain means basically gives an idea of how much smaller the power of $e[n]$ is compared to $x[n]$, so a large G_p means that prediction error has significantly lower power than the original signal, indicating that the predictor is working well.

Part E – Effect of Predictor Coefficient Using MATLAB

For $a = 0, 0.5, 0.8, 1, 1.2$

After calculating MSE for each coefficient, we got these values:

```
Command window

MSE(0) = 2.104271

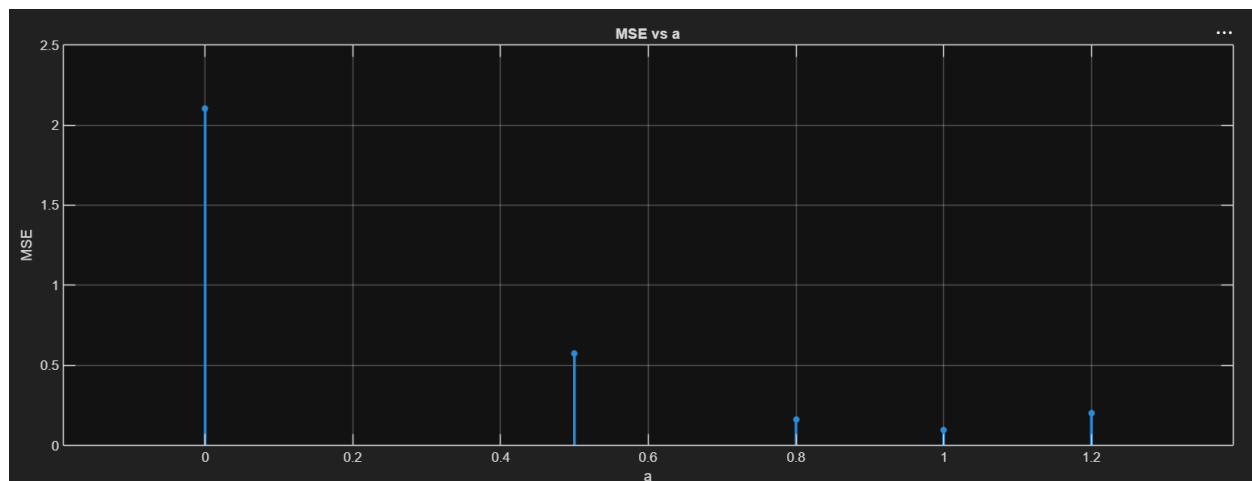
MSE(0.5) = 0.574035

MSE(0.8) = 0.161625

MSE(1) = 0.097407

MSE(1.2) = 0.201767
>>
```

And here is what the plot looks like:



Questions (8 - 11):

8. Which value gives the minimum MSE?

We get the minimum MSE when $a = 1$.

9. Is $a = 1$ necessarily the optimum predictor?

No, $a = 1$ is the best coefficient out of the values given, but we could calculate an even better coefficient (optimum predictor coeff.) to give us the least MSE possible.

10. What determines the optimum predictor coefficient?

It's determined by minimizing the prediction MSE. The value of a that produces the smallest MSE is considered the optimum coefficient.

11. What happens when the predictor coefficient is poorly selected?

When the predictor coefficient is poorly selected, the predicted signal is less similar to the original signal. This increases the prediction error $e[n]$, which increases the MSE, resulting in a poorer prediction.

Part F – Calculate the Optimum Predictor in MATLAB

After calculating a_{opt} and finding MSE we get these values:

```
Command Window

MSE(a_opt) = 0.096213

MSE(1) = 0.097407

Difference = 0.001195
>>
```

So a_{opt} has a 1.22% improvement over $a = 1$.

Question:

Calculate the prediction error using a_{opt} and compare its MSE with the MSE for $a = 1$. Is the difference significant? Explain based on the characteristics of the signal.

Answer:

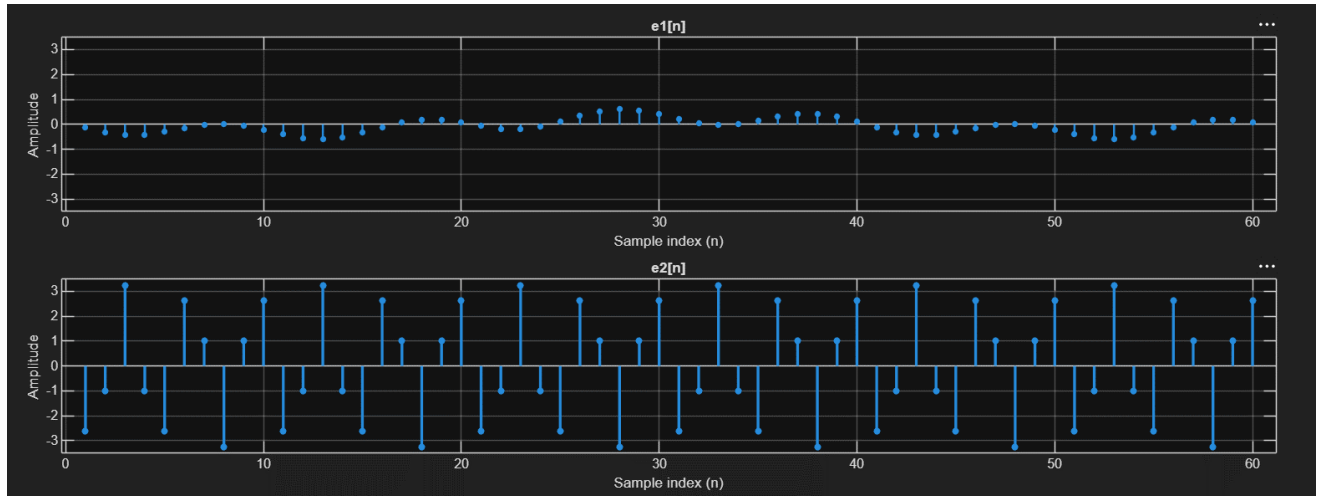
No, the difference is not significant. This is because the signal changes gradually and has strong correlation between adjacent samples. Therefore, using $x[n-1]$ with $a = 1$ already provides a good prediction, so optimizing the coefficient only produces a small improvement in the MSE.

Part G – Effect of Signal Frequency

After using the simple predictor for the signal $x_2[n] = 2 \cos(0.6\pi n)$,

And also plotting the first signal's error $x[n] = 2 \cos(0.05\pi n) + 0.5 \cos(0.2\pi n)$.

We got those prediction error plots:



We should also compare the Prediction Gain:

```
Command Window
First Signal:
Gp = 21.815616
Gp(dB) = 13.387675

Second Signal:
Gp = 0.382560
Gp(dB) = -4.173004
>>
```

From the G_p alone, it is obvious that the first signal performs better for DPCM.

Questions:

12. Which signal is easier to predict using the previous sample?

As we can see from the plots the first signal (with lower frequencies) is much easier to predict.

This is also confirmed by the prediction gain, which is positive for the first signal and negative for the second.

13. Why?

Because it has low frequencies, causing strong correlation between adjacent samples, which makes the simple predictor that depends on previous samples ($\hat{x}[n] = x[n-1]$) produce better results, making the difference between the sample and the prediction minimal, hence small prediction error.

14. What does this demonstrate about the relationship between signal correlation and DPCM performance?

It demonstrates that DPCM performance depends heavily on the signals correlation. Strong correlation between samples, leads to smaller error, allowing DPCM to use less bits for quantization.

(The stronger the correlation, the better DPCM performs).

Part H solution :

(15) The receiver only gets the quantized error signal $x - \hat{q}[n] = x^n[n] + e - \hat{q}[n]$ which means it needs its own copy of the predictor to generate $x^n[n]$ from previously reconstructed samples. Without matching predictor, the receiver has no way to turn the error signal back into the actual waveform.

(16) if the predictors differ even slightly, then $x^n[n]$ computed at the transmitter (used to form $e[n]$) won't match the $x^n[n]$ computed at the receiver (used to reconstruct $x - \hat{q}[n]$). This mismatch introduces an additional, uncontrolled error beyond the quantization error, causing the reconstructed signal to drift away from the original. The whole DPCM scheme relies on both sides tracking the same prediction.

(17)

The receiver never has access to the original samples $x[n]$ - it only ever has the reconstructed samples $x_q[n]$. If the transmitter based its prediction on the true $x[n-1]$, its $\hat{x}[n]$ would differ from what the receiver can compute (since the receiver only has $x_q[n-1]$). By using $x_q[n-1]$ at the transmitter too, both sides work from identical information keeping the predictors synchronized.

(18)

~~What if~~ This causes a mismatch between transmitter and receiver, and the prediction errors don't cancel correctly during reconstruction. Small discrepancies can accumulate over time (error propagation/drift), leading to a growing reconstruction error that isn't due to quantization at all - potentially causing the receiver's estimate of the signal to diverge significantly from the true signal.

(19) Speech signals are highly correlated from one sample to next one (they change relatively slowly within short time windows due to the physical constraints of vocal tract movement). This ~~strong~~ strong sample-to-sample correlation means a predictor can estimate $x[n]$ quite accurately from past samples. So the prediction error $e[n]$ has much smaller amplitude / variance than $x[n]$ itself allowing it to be quantized with fewer bits for same quality, i.e. DPCM exploits redundancy that's abundant in speech.

(20) No, if the samples are completely uncorrelated knowing $x[n-1]$ gives no information about $x[n]$. The best predictor would essentially give $x[n] \approx 0$ (or constant) meaning $e[n] \approx x[n]$ - the prediction error has the same power / range as the original signal. In this case DPCM provides no dynamic range reduction, a net no coding gain over straight PCM; the predictor simply cannot exploit correlation that isn't there.

Bonus - Second-Order Predictor

Model

The second-order predictor is defined as:

$$\hat{x}[n] = a_1 x[n-1] + a_2 x[n-2]$$

Using the same sequence from Part A: $x[n] = \{10, 12, 15, 17, 18, 20, 21, 23\}$, $n = 0, \dots, 7$

Deriving the Optimum Coefficients

To minimize the squared prediction error $\sum e^2[n]$, the normal equations are solved:

$$a_1 \sum x^2[n-1] + a_2 \sum x[n-1]x[n-2] = \sum x[n]x[n-1]$$

$$a_1 \sum x[n-1]x[n-2] + a_2 \sum x^2[n-2] = \sum x[n]x[n-2]$$

Summation Table

Quantity	Value
$\sum x^2[n-1]$	1823
$\sum x^2[n-2]$	1482
$\sum x[n-1]x[n-2]$	1641
$\sum x[n]x[n-1]$	2004
$\sum x[n]x[n-2]$	1802

This gives the system:

$$1823 a_1 + 1641 a_2 = 2004$$

$$1641 a_1 + 1482 a_2 = 1802$$

Solving: $a_1 \approx 1.4589$, $a_2 \approx -0.3995$

Prediction Error and MSE

n	x[n]	$\hat{x}[n]$	e[n]
2	15	13.51	1.49
3	17	17.09	-0.09
4	18	18.81	-0.81
5	20	19.47	0.53
6	21	21.99	-0.99
7	23	22.65	0.36

$$\text{MSE}_2 \approx 0.7097$$

Comparison with First-Order Optimum Predictor

Using $a_{\text{opt}} = 1.1045$ from Part C, restricted to the same range ($n = 3:N$) for a fair comparison:

$$\text{MSE}_1 \approx 0.8465$$

Since $\text{MSE}_2 < \text{MSE}_1$, the second-order predictor achieves a lower prediction error than the optimum first-order predictor.

MATLAB Verification

The MATLAB output confirming the calculated coefficients and MSE values is shown below.

```
a_opt for first order predictor:
    1.1045

a1 = 1.4589 , a2 = -0.3995
MSE first order (n=3:N) = 0.8465
MSE second order = 0.7097
```

Figure 1: MATLAB command window output showing a_{opt} , a_1 , a_2 , and MSE values.

Plots

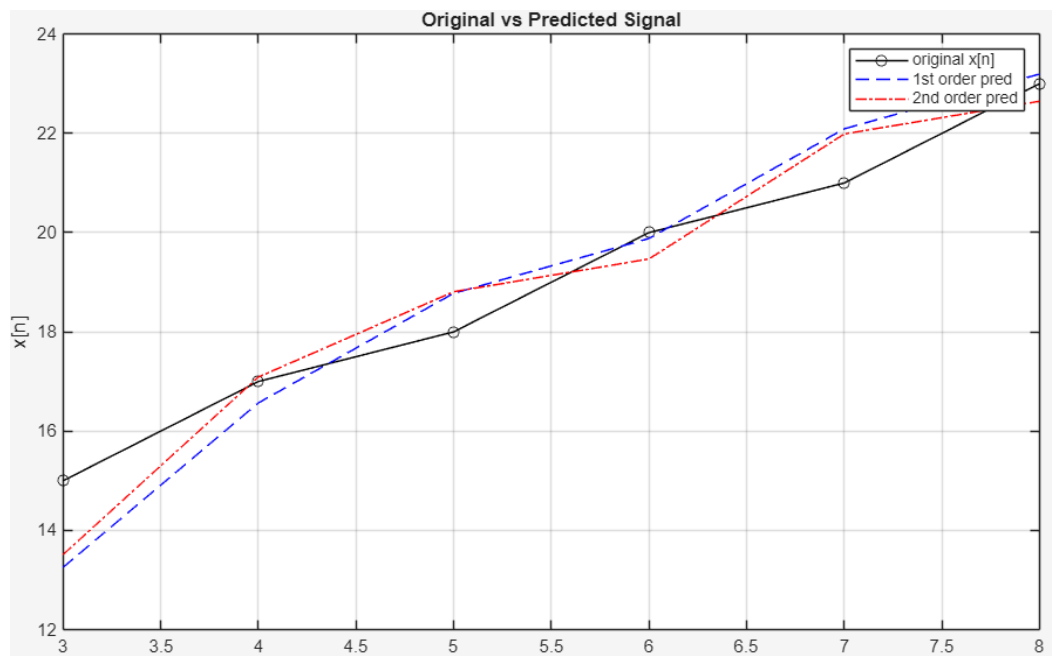


Figure 2: Original signal compared with first-order and second-order predictions.

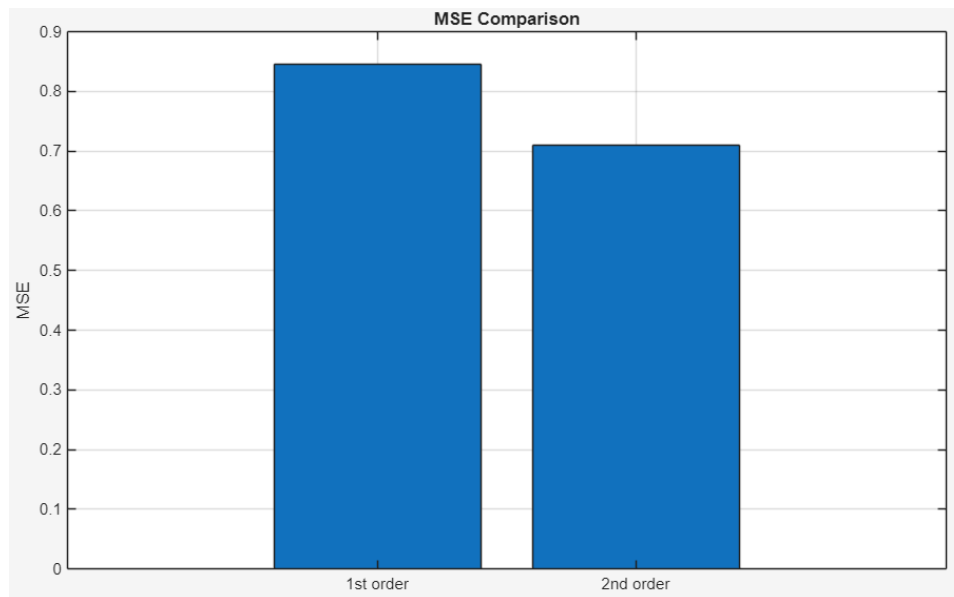


Figure 3: MSE comparison between first-order and second-order predictors.

Discussion and Conclusion

As shown in Figure 3, the second-order predictor achieves a lower MSE (≈ 0.71) compared to the optimum first-order predictor (≈ 0.85), an improvement of roughly 16%. This confirms that using two previous samples allows the predictor to exploit additional correlation in the signal.

However, increasing the predictor order is not always beneficial. Higher-order predictors generally reduce MSE further, but the improvement shows diminishing returns as the order increases, while each additional order requires an extra coefficient and additional multiply-accumulate operations per sample, which is costly in real-time systems such as speech coding. More coefficients also increase sensitivity to estimation errors, especially with limited or noisy data, and raise the risk of overfitting, and they must be stored and computed while additional past samples are kept in memory, increasing memory usage and delay. Taken together, increasing the predictor order improves performance only up to a point, governed by how much correlation actually exists between distant samples; beyond that point, the marginal reduction in MSE becomes small relative to the added complexity and sensitivity, so in practice, low-order predictors (order 2-4) are typically used as a practical trade-off.

DPCM Predictor Assignment

Part A — First-Order Prediction ($\hat{x}[n] = x[n-1]$)

Sequence: $x[n] = \{10, 12, 15, 17, 18, 20, 21, 23\}$

n	0	1	2	3	4	5	6	7
$x[n]$	10	12	15	17	18	20	21	23
$\hat{x}[n]$	-	10	12	15	17	18	20	21
$e[n]$	-	2	3	2	1	2	1	2

Range of $x[n]$: $23 - 10 = 13$. Range of $e[n]$: $3 - 1 = 2$. The prediction error occupies a much smaller dynamic range than the original signal, so it can be quantized with fewer bits for the same step size, or with a finer step size using the same number of bits.

Part B — Predictor Coefficient MSE Comparison Table

Sequence used: $x[n] = \{10, 12, 15, 17, 18, 20, 21, 23\}$. Predictor: $\hat{x}[n] = a \cdot x[n-1]$

Predictor coefficient, a	Prediction error $e[n]$	Prediction MSE
0	12, 15, 17, 18, 20, 21, 23	336.00
0.5	7, 9, 9.5, 9.5, 11, 11, 12.5	101.25
1	2, 3, 2, 1, 2, 1, 2	3.857
1.2	0, 0.6, -1, -2.4, -1.6, -3, -2.2	3.360

Among the tested fixed values, $a = 1.2$ gives the smallest MSE (3.360), slightly better than $a = 1$ (3.857). This shows that the true optimum lies near, but not exactly at, $a = 1$ for this particular sequence.

Part C — Optimum Predictor Coefficient

$$a_{\text{opt}} = (\sum x[n]x[n-1]) / (\sum x^2[n-1]) = 2124 / 1923 \approx 1.1045$$

Predictor	a	MSE
$a = 0$	0	336.00
$a = 0.5$	0.5	101.25
$a = 1$	1	3.857
$a = 1.2$	1.2	3.360
a_{opt} (Part C)	1.1045	0.856

The optimum coefficient $a_{\text{opt}} \approx 1.1045$ produces a dramatically lower MSE (0.856) than any of the fixed values tested in Part B, confirming that least-squares optimization of the predictor coefficient gives the best achievable first-order prediction for this sequence.

Part E — Predictor Coefficient MSE (MATLAB Signal)

Signal: $x[n] = 2\cos(0.05\pi n) + 0.5\cos(0.2\pi n)$, $n = 0, \dots, 199$. Predictor: $\hat{x}[n] = a \cdot x[n-1]$

a	0	0.5	0.8	1	1.2
MSE	2.1043	0.5740	0.1616	0.0974	0.2018

Among the tested values, $a = 1$ gives the minimum MSE (0.0974). Since this signal is a sum of slowly varying cosines, consecutive samples are strongly correlated and close to equal, so the simple predictor $\hat{x}[n] = x[n-1]$ ($a = 1$) already performs very well; $a = 1.2$ overshoots and increases the error again.

Part F — Optimum Predictor Coefficient (MATLAB Signal)

$a_{\text{opt}} = \Sigma(x[n]x[n-1]) / \Sigma(x^2[n-1]) \approx 0.9762$ (computed over $n = 1, \dots, 199$)

Predictor	a	MSE
$a = 1$ (simple predictor)	1	0.09741
a_{opt} (Part F)	0.9762	0.09621

The difference between MSE at $a = 1$ (0.09741) and MSE at a_{opt} (0.09621) is very small (about 1.2%). This is because the signal is smooth and highly correlated (dominated by a slowly varying cosine component), so the simple predictor $a = 1$ is already very close to optimal; the least-squares correction offers only a marginal improvement for this type of signal.

Bonus — Second-Order Predictor Comparison

Predictor: $\hat{x}[n] = a_1 x[n-1] + a_2 x[n-2]$, with $a_1 \approx 1.4589$, $a_2 \approx -0.3995$ (verified in MATLAB).

Predictor	Coefficients	MSE
1st-order optimum ($n = 3:N$)	$a_{\text{opt}} = 1.1045$	0.8465
2nd-order optimum	$a_1 = 1.4589$, $a_2 = -0.3995$	0.7097

The second-order predictor reduces the MSE from 0.8465 (first-order optimum) to 0.7097, an improvement of about 16%, confirming that adding one more past sample can further exploit the correlation present in this sequence, though with added computational cost.

Final Conclusion

In a DPCM system, the predictor uses previously reconstructed samples to estimate the value of the current sample before it is transmitted, so that only the difference between the actual and predicted value — the prediction error $e[n]$ — needs to be quantized and sent. Because real-world signals such as speech are highly correlated from sample to sample, a well-designed predictor can estimate the next sample quite accurately, which makes $e[n]$ much smaller in amplitude and dynamic range than the original signal $x[n]$. This smaller dynamic range means fewer quantization levels, and therefore fewer bits, are needed to represent $e[n]$ at the same quality as directly quantizing $x[n]$. A good predictor, whether chosen empirically or derived through least-squares optimization as in Part C, minimizes the prediction error variance and therefore maximizes the prediction gain, directly improving the coding efficiency of the DPCM system without sacrificing reconstruction quality.